

BAIL security

Terminal Finance Extension

FINAL REPORT

April ‘2025

Disclaimer:

Security assessment projects are time-boxed and often reliant on information that may be provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

The content of this assessment is not an investment. The information provided in this report is for general informational purposes only and is not intended as investment, legal, financial, regulatory, or tax advice. The report is based on a limited review of the materials and documentation provided at the time of the audit, and the audit results may not be complete or identify all possible vulnerabilities or issues. The audit is provided on an "as-is," "where-is," and "as-available" basis, and the use of blockchain technology is subject to unknown risks and flaws.

The audit does not constitute an endorsement of any particular project or team, and we make no warranties, expressed or implied, regarding the accuracy, reliability, completeness, or availability of the report, its content, or any associated services or products. We disclaim all warranties, including the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

We assume no responsibility for any product or service advertised or offered by a third party through the report, any open-source or third-party software, code, libraries, materials, or information linked to, called by, referenced by, or accessible through the report, its content, and the related services and products. We will not be liable for any loss or damages incurred as a result of the use or reliance on the audit report or the smart contract.

The contract owner is responsible for making their own decisions based on the audit report and should seek additional professional advice if needed. The audit firm or individual assumes no liability for any loss or damages incurred as a result of the use or reliance on the audit report or the smart contract. The contract owner agrees to indemnify and hold harmless the audit firm or individual from any and all claims, damages, expenses, or liabilities arising from the use or reliance on the audit report or the smart contract.

By engaging in a smart contract audit, the contract owner acknowledges and agrees to the terms of this disclaimer.

1. Project Details

Important:
Please ensure that the deployed contract matches the source-code of the last commit hash.

Project	Terminal - Extension
Website	
Language	Solidity
Methods	Manual Analysis
Github repository	Last audited commit: https://github.com/terminal-finance/contracts/tree/3ef5a12896cecb2f3298a7f1fd889a5e566a2db4/contracts Extension 1: https://github.com/terminal-finance/contracts/tree/c0bdb8e0a1c583d6a07900e777ae66456c44500b/contracts * **VotingEscrow.sol** * **HistoricalTokenBalance.sol** * **Voter.sol** * MerkleClaim.sol * RewardsDistributor * GoVeTerm * HistoricalSupply Extension 2: https://github.com/terminal-finance/contracts/tree/486a65e23b0d6ee710baa0d0af4e0c758d24da7d/contracts * /operator/Operator.sol * /operator/Payout.sol * /bribes/BribeScheduler.sol
Resolution 1	https://github.com/terminal-finance/contracts/tree/26caa26210670176e55257f8df905aa4f140dfd5/contracts

2. Detection Overview

Severity	Found	Resolved	Partially Resolved	Acknowledged (no change made)	Failed resolution
High	2	2			
Medium	2			2	
Low	12	4		8	
Informational	3	1		2	
Governance	2			2	
Total	21	7		14	

2.1 Detection Definitions

Severity	Description
High	The problem poses a significant threat to the confidentiality of a considerable number of users' sensitive data. It also has the potential to cause severe damage to the client's reputation or result in substantial financial losses for both the client and the affected users.
Medium	While medium level vulnerabilities may not be easy to exploit, they can still have a major impact on the execution of a smart contract. For instance, they may allow public access to critical functions, which could lead to serious consequences.
Low	Poses a very low-level risk to the project or users. Nevertheless the issue should be fixed immediately
Informational	Effects are small and do not post an immediate danger to the project or users
Governance	Governance privileges which can directly result in a loss of funds or other potential undesired behavior

3. Detection

The following files have been changed:

Voter.sol: https://www.diffchecker.com/3IJKNsDu/

VotingEscrow.sol: https://www.diffchecker.com/T8UfttG9/

MerkleClaim.sol: https://www.diffchecker.com/H01aOgbK/

RewardsDistributor: https://www.diffchecker.com/OhblxEtU/

GoVeTerm: https://www.diffchecker.com/ZejQNegl/

HistoricalTokenBalance: https://www.diffchecker.com/ysNtl6Sy/

HistoricalSupply: https://www.diffchecker.com/00joW27T/

VotingEscrow

The following section will be considered as a differential audit between the latest audited commit and the new commit. Differential audits are inherently limited as they only focus on the changes.

The following changes have been identified:

a) Implementation of operatorLock
b) Implementation of permanent locks
c) _checkpoint adjustment to reflect attachment/detachment and permanent lock logic
d) Adjustment of various VP getter functions for operated balance
e) Attachment/detachment of tokenId to/from operatorLock
f) Adjustment of increaseAmount for permanent and operatorLock
g) Adjustment of merge for permanent lock and restrictions
h) Adjustment of split for restrictions
i) Adjustment of withdraw for restrictions

Core Invariants:

INV 1: Only governance can create an operatorLock

INV 2: An initial operatorLock must have zero VP

INV 3: During attachment, checkpoint of tokenId must get operatedBalance assigned to amount and operatorTokenId to corresponding operator ID

INV 4: During detachment, checkpoint of tokenid must get operatedBalance set to zero and operatorTokenId to zero

INV 5: During conversion to permanent/increaseAmount of permanent lock/merge to permanent lock/adjustment of operator lock, checkpoint from newLocked must always be marked with permanentBalance being amount

INV 6: Any _checkpoint call with non-zero tokenId must set global.permanentBalance to permanentLockBalance

INV 7: votingPowerAtWithOperatedBalance must only be consulted during governance votes

INV 8: A normal created lock must always be non-permanent

INV 9: A operator lock must always be permanent

INV 10: Attaching a non-permanent lock must increase permanentLockBalance

INV 11: Attaching a lock must set normal VP for tokenId to zero

INV 12: Detaching a lock must decrease permanentLockedBalance and create maximum lock for tokenId

INV 13: increaseAmount for permanent lock must increase permanentLockBalance

INV 14: increaseAmount for operatorLock must call operator.notifyLockedRevenue

INV 15: increaseUnlockTime must not be allowed for permanent/attached lock

INV 16: merge is not allowed for permanent/attached/operator locks

INV 17: split is not allowed for permanent/attached locks

INV 18: withdraw is not allowed for permanent/attached locks

INV 18: expired lock cannot be converted to permanent lock

INV 19: attached lock cannot be converted to permanent lock

INV 20: operator lock cannot be converted to normal lock

INV 21: Only permanent lock can be converted to normal lock

INV 22: attached lock cannot be converted to permanent lock

Issue_01	Governance: Skew governance votes by creating lock to another operator
Severity	**Governance**
Description	Currently, governance of this contract has several privileges for invoking certain functions that can drastically alter the contracts behavior. For example, an operator lock can be transferred to a custom operator contract which then returns incorrect locked revenue for attached tokenIds.
Recommendations	Consider incorporating a Gnosis Multisignature contract as owner and ensuring that the Gnosis participants are trusted entities.
Comments / Resolution	Acknowledged.

Issue_02	createOperatorLockFor() can fail due to MAX_TOKEN_DELEGATIONS griefing
Severity	**Low**
Description	Any address that holds a VotingEscrow lock is subject to the MAX_TOKEN_DELEGATIONS limit, which can be reached maliciously if an attacker transfers or creates multiple dust locks on behalf of a target. In the new Operator system, the intended design is for the Operator contract to hold a single dedicated operator lock, minted via createOperatorLockFor() in the VotingEscrow contract. However, there is currently no safeguard to ensure that the Operator contract has not already been griefed with dust locks before createOperatorLockFor() is called. If the delegation limit has already been reached, the mint would fail. In this case, the only solution would likely be to redeploy the Operator contract and try again, as the Operator lacks any mechanism to discard unwanted locks. It's also worth noting that this griefing vector can be an issue even if the Operator deployment and createOperatorLockFor() call are atomic, since it's possible to preemptively transfer tokens to a contract address before any code is deployed.
Recommendations	Consider adding a mechanism to atomically remove unwanted locks from the Operator before minting, similar to MerkleClaim. Alternatively, ensure the deployment addresses are unknown (such as by using a private mempool and atomic deployment with createOperatorLockFor()) to make griefing attempts unlikely.
Comments / Resolution	Acknowledged.

Issue_03	Attaching/detaching is not possible in first hour due to limitation in poke
Severity	**Low**
Description	Attaching always pokes the operator lock: *// Notify the voter of the updated voting powers* *IVoter(voter).attachToOperatorLock(tokenId, operatorTokenId);* This will not work in the first hour because poking is limited: *if (block.timestamp % DURATION < RESTRICTED_VOTING_PERIOD) {* *revert RestrictedVotingPeriod();* *}*
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_04	Attached tokenId is transferable
Severity	**Low**
Description	Currently, it is possible to transfer tokenIds which are attached. This allows for griefing the MerkleClaim contract and potentially other issues. Most of the time, exploits happen due to arbitrary user inputs or users invoking functions which are not meant to be invoked by users, one can argue that a large user flexibility is a great seed for exploits. Therefore, at BailSec, we are of the opinion that codebases should never provide more user flexibility than necessary during the normal business logic.
Recommendations	Consider not allowing the transfer of attached tokenIds.
Comments / Resolution	Revolved.

Issue_05	Inconsistency in storage update
Severity	**Informational**
Description	In the legacy codebase, all lockedBalances[id] updates were done before any checkpoint update. This is not consistent with the way how: - convertToPermanentLock - convertToNormalLock - detachFromOperatorLock are handling these updates.
Recommendations	Consider staying consistent.
Comments / Resolution	Resolved.

HistoricalTokenBalance

The following section will be considered as a differential audit between the latest audited commit and the new commit. Differential audits are inherently limited as they only focus on the changes.

The following changes have been identified:

a) Renaming from IVotingEscrow.Point to IVotingEscrow.TokenPoint
b) Implementation of permanentBalance into individual VP
c) Implementation of operatedBalance into individual VP

Core Invariants:

INV 1: If a lock is permanent, return permanentBalance as VP

INV 2: If a lock is attached the function fetches operatedBalance, return operatedDeposit + revenue

No issues found.

Voter

The following section will be considered as a differential audit between the latest audited commit and the new commit. Differential audits are inherently limited as they only focus on the changes.

The following changes have been identified:

a) Implementation of RESTRICTED_VOTING_PERIOD:
    i) Voting not allowed within the first hour
    ii) Attaching not allowed within the last hour
    iii) Attaching/Detaching not allowed within the first hour
    iv) Poking not allowed within the first hour
    v) Voting only allowed during the last hour for operator locks
b) Implementation of attachToOperatorLock
c) Implementation of detachFromOperatorLock

Core Invariants:

INV 1: Voting is not allowed within the first hour of an epoch

INV 2: Voting with non-operator lock is not allowed within last hour of an epoch

INV 3: Voting with attached tokenId will always have zero VP

INV 4: Attaching must reset attached lock and poke operator lock

INV 5: Detaching must reset or poke operator lock

INV 6: Attaching must set lastVoted[tokenId]

INV 7: Poke is not allowed within the first hour of an epoch

No issues found.

HistoricalSupply

The following section will be considered as a differential audit between the latest audited commit and the new commit. Differential audits are inherently limited as they only focus on the changes.

The following changes have been identified:

a) Implementation of permanentBalance in global VP

Core Invariants:

INV 1: The totalVotingPowerAt function must return the current VP including the permanent VP

No issues found.

MerkleClaim

The following section will be considered as a differential audit between the latest audited commit and the new commit. Differential audits are inherently limited as they only focus on the changes.

The following changes have been identified:

a) Incorporation of additional values within the lockedBalances return value

Issue_06	MerkleClaim can be bricked by unmergeable locks
Severity	**High**
Description	Among the new VotingEscrow features, two additional lock types have been introduced: permanent locks (which do not have an expiration date) and attached locks (which are associated with an operator lock). Both of these lock types are subject to restrictions when using the merge() function. Specifically, permanent locks cannot be used as the "from" token in a merge, and attached locks cannot be used as either token. Both types of locks remain freely transferable between addresses. Since lock owners are subject to the MAX_TOKEN_DELEGATIONS limit, users need to be aware of potential DoS attacks in which griefers send many small dust locks to their address, pushing them to the limit. Normally, users can mitigate this by either burning the tokens (e.g. transferring them to another address) or merging them. However, because permanent and attached locks cannot be merged, using merging as a mitigation is no longer always possible. This is particularly relevant for the MerkleClaim contract, which is designed to handle this type of DoS by merging two of its locks when needed. If a griefer sends permanent or attached locks to the MerkleClaim contract, any attempt to merge them will revert, causing the claim to fail. This effectively freezes the locked claiming logic and prevents users from completing their claims.

	Notably, it doesnt even require to send 128 tokenIds, as it will already revert if there are >= 3 tokenIds, due to the inherent merge attempt.
Recommendations	To eliminate this DoS vector, consider modifying the MerkleClaim contract to burn any unwanted tokenIds by transferring them to address(0) instead of attempting to merge them. Furthermore, we recommend limiting the transfer of attached tokenIds within the VE.
Comments / Resolution	Resolved, if a permanent lock is detected as the “from” token for a potential merge, it is first converted to a normal token and then merged. If a permanent token is the “to” token, merge will still work.

Issue_07	Pre-locking of tokens will influence proposal threshold for governance votings
Severity	**Low**
Description	Currently, the contract stores a tokenId with potentially significant VP. This VP cannot be used for governance votes while it is stored within the MerkleClaim contract. Therefore it can be considered as dead VP which significantly alters governance votes.
Recommendations	Consider acknowledging this issue
Comments / Resolution	Acknowledged.

Issue_08	DoS of contract before start is called
Severity	**Low**
Description	The contract expects to receive one tokenId which represents all unclaimed locks. The transfer of this tokenId within the start function can revert if the contract already holds 128 tokenIds. A malicious user can simply transfer 128 tokenIds to the MerkleClaim contract which then provokes a revert upon transferFrom during start. This can even be done before the contract is deployed.
Recommendations	Consider acknowledging this issue and re-deploy the contract using a private mempool if this issue gets exploited.
Comments / Resolution	Acknowledged.

RewardsDistributor

The following section will be considered as a differential audit between the latest audited commit and the new commit. Differential audits are inherently limited as they only focus on the changes.

The following changes have been identified:

a) Return value of tokenPointHistory is adjusted to reflect new struct
b) Prevent attached tokenId from claiming
c) Return value of lockedBalances is adjusted to reflect new struct
d) permanent check in claim included to prevent bug which allows permanent locks to claim liquid

Core Invariants:

INV 1: Attached tokenIds are not allowed to claim rewards
No issues found.

GoVeTerm

The following section will be considered as a differential audit between the latest audited commit and the new commit. Differential audits are inherently limited as they only focus on the changes.

The following changes have been identified:

a) Individual VP must be fetched via votingPowerAtWithOperatedBalance

Core Invariants:

INV 1: VP must include permanentBalance

INV 2: VP must include operatedBalance and revenue

No issues found.

EXTENSION 2

Operator

The Operator contract is the custodian of the operator lock. This contract handles different functionalities on behalf of the tokenId such as voting, claiming rewards and claiming voting revenue.
Furthermore, swap functionality was introduced which allows for swapping voting rewards freely around.
Any rewards from the RewardsDistributor are simply compounded back into the tokenId value which is then distributed among all attached tokenIds, proportionally to their share in the previous epoch.

Appendix: totalActiveBalance

The totalActiveBalance variable accurately tracks the active balance in the last epoch. It includes any attached balance plus claimable balance from rewards which are compounded back into the tokenId, but not these from the current epoch

Appendix: Reward Mechanism

Whenever rewards are claimed, these will be compounded back into the tokenId via VotingEscrow.increaseAmount which will then in turn call the notifyLockedRevenue function. The reward amount is then divided by totalActiveBalance which simply reflects the underlying balance from the last epoch which is entitled to receive rewards. This is the shared VP of all attached tokenIds including any unclaimed rewards from past epochs.

Appendix: Whitelist

The contract employs a whitelist which determines the addresses which are allowed to attach their tokenIds to the operator. The possibility to completely disable the whitelist exists which then in turn allows anyone to attach their tokenId.

Appendix: balanceAt

The _balanceAt function is responsible for fetching the balance of a tokenId at a specific timestamp. To achieve that, the following algorithm is applied:

> Execute binary search to find the latest session before or at the search timestamp

> Optional: Return early if the tokenId was detached before the search timestamp

> If already updated: Return balanceHistory at the search timestamp

> If not updated: Apply rewards during a looping scheme until the search timestamp

Core Invariants:

INV 1: _activeSessionOf must return most recent session

INV 2: _activeSessionOf must revert if a tokenId has no session

INV 3: _activeSessionOf must revert if session.timeEnd is non-zero

INV 4: sendRevenue/notifyAttach/notifyDetach/notifyLockedRevenue must update epoch at the beginning of the function

INV 5: Only whitelisted addresses can attach a tokenId if whitelist is enabled

INV 6: updateCheckpointOf can only be called with epochNumber = _epochNumber -1

INV 7: The sum of all open sessions' balance (at current epoch) must be equal to totalBalance

INV 8: The sum of all open sessions' balance (at current epoch -1) must be equal to totalActiveBalance

Privileged Functions

- initialize
- setName
- setTag
- setGuardian
- whitelist
- disableWhitelist
- vote
- collectRevenue
- swapRevenue
- sendRevenue
- lockRevenue

Issue_09	Governance Issue: Extraction of revenue via slippage
Severity	**Governance**
Description	Currently, governance of this contract has several privileges for invoking certain functions that can drastically alter the contracts behavior. For example, it is possible to swap via a low liquidity pool to extract revenue.
Recommendations	Consider incorporating a Gnosis Multisignature contract as owner and ensuring that the Gnosis participants are trusted entities.
Comments / Resolution	Acknowledged.

Issue_10	Incorrect _balanceAt() revenue accrual for mid-epoch timestamps can distort governance voting calculations
Severity	**Medium**
Description	The second return value of the _balanceAt() function in the Operator contract is intended to represent the total balance (including accrued revenue) of a given attached tokenId up to a specific timestamp. To compute this, _balanceAt() calculates the epochNumber as timestamp / 1 weeks and passes this to _finalBalanceAt(). The _finalBalanceAt() function then accumulates all revenue up to and including that epoch. However, this includes the entire final epoch, even revenue accrued after the target timestamp but still within the same epoch, which does not match the intended use case of the function. This is documented in the NATSPEC: */// @dev Internal function to retrieve the balance of a session at* */// the end of a given epoch. This function assumes that given session* However, this issue can affect other parts of the system, such as the HistoricalTokenBalance library, where _votingPowerAt() may return inflated results by including revenue from the same epoch but after the target timestamp. The issue is especially concerning for governance votes, as the total voting power (used as the denominator in vote weight calculations) is not affected in the same way. This inconsistency allows operator locks to gain extra voting power relative to the total and distort voting power calculations. This inconsistency will arise in a very specific edge-case: If increaseAmount was called *after* the timestamp where the VP is consulted. In that scenario, the increase of permanentBalance will not flow into the global VP. However, the revenue will still fully flow into it, which effectively results in the sum of all individual VPs becoming larger than the global VP.

	Contrary to that, if increaseAmount is called before the VP consulting timestamp but in the middle of an epoch, the global VP will include the increase of permanentBalance and at the same time the extrapolation of the locked revenue until the epoch end will result in an exact match of the global VP and the sum of all individual VPs.
Recommendations	To mitigate this issue in the governance voting calculations, one approach is to require that vote timestamps align exactly with the end of an epoch. This ensures there's no period where revenue accrues after the voting timestamp but still within the same epoch, preventing the mismatch. Alternatively, consider modifying _finalBalanceAt() to exclude all revenue from the epoch containing the timestamp. While this would now undercount the actual voting power (it would ignore revenue in the epoch that actually was from before the target timestamp), it is less dangerous than the current behavior of overcounting voting power. It has to be noted that notifyDetach would still allow the user to claim the full revenue for a running epoch which then in turn allows the user to vote with the original tokenId once it is unattached. A possible fix would be to not allow a user to vote/merge/split in the epoch where a tokenId has been detached. The team should carefully select the fix with the smallest code changes and least side-effects. Alternatively, this issue can also be acknowledged with a note for that behavior, if changes are too intrusive and risk further side-effects.
Comments / Resolution	Acknowledged.

Issue_11	Revenue is immediately claimable in same epoch
Severity	**Medium**
Description	Similar to the above mentioned issue which is related to the fact that the _finalBalanceAt function includes the full revenue of the current epoch, a detach action also allows for claiming the full revenue of the current epoch. If the previous issue is fixed, a user could then still detach and at the timestamp of the detach, this tokenId will include the full revenue (while it should not include it). Furthermore, with the fact in mind that rewards which are notified in epoch N can be claimed in epoch N during notifyDetach, an inconsistency arises if a tokenId is detached before rewards are notified, as any incoming rewards will then be distributed among all other tokenIds.
Recommendations	Consider not allowing to claim the revenue for the current epoch.
Comments / Resolution	Acknowledged.

Issue_12	DoS due to direct transfers before operatorToken is minted
Severity	**Low**
Description	Currently, it is possible to transfer 128 tokenIds to the Operator contract which would then revert upon initialization whenever the operatorId is transferred in.
Recommendations	Consider acknowledging this issue and redeploying the contract in that scenario.
Comments / Resolution	Acknowledged.

Issue_13	Lack of SafeERC20 usage
Severity	**Low**
Description	The contract uses the standard ERC20 pattern for transfers. This will malfunction for tokens that do not follow the IERC20 interface, for example those that return false or do not return a boolean on the transfer/approve call. ... *IERC20(token).transfer(payout, amount);* *IPayout(payout).notifyPayout(token, amount);* ...
Recommendations	*Consider using safeTransfer.*
Comments / Resolution	Resolved.

Issue_14	Redeemable tokens can never be swapped
Severity	**Low**
Description	Currently, the contract does not expose functionality to swap redeemable tokens. This is an inherent limitation of the underlying DEX.
Recommendations	Consider implementing a function which allows for unwrapping redeemable tokens to the underlying token. This function must be callable by the guardian only.
Comments / Resolution	Resolved.

Issue_15	Incorrect setting of totalBalanceHistory in case of non-updated/skipped epoch
Severity	**Low**
Description	The flipEpochIfNecessary function sets totalBalanceHistory for the previous epoch: *totalBalanceHistory[currentEpochNumber - 1] = totalBalance;* In the scenario where one epoch is skipped due to a lack of update, totalBalanceHistory for this epoch will be zero while in reality an existing balance was present.
Recommendations	Consider ensuring that flipEpochIfNecessary is at least updated once per epoch.
Comments / Resolution	Acknowledged.

Issue_16	Deviation from trusted binary search implementation
Severity	**Low**
Description	Every trusted binary search implementation works as follows: a) Fetch index of lower and upper element b) Calculate the center: > center = upper - (upper - lower) / 2 c) Return early if the center TS matches the search timestamp OR set lower = center if timestamp is above center TS OR set upper = center -1 if timestamp is below center TS The current implementation does not return early if the center TS matches the search timestamp which can not only have gas implications but can expose a vulnerability in edge-cases.
Recommendations	Consider following the trusted implementation as well as implementing fuzzing.
Comments / Resolution	Resolved.

Payout

The Payout contract is in direct connection with the Operator, as it is being deployed upon Operator deployment and permanently tied to the Operator contract.

Once or multiple times per epoch, the Payout contract receives revenue from the Operator which is then allocated towards attached tokenIds. Authorized addresses for tokenIds can then claim any revenue which was previously allocated.

Appendix: Revenue Distribution

The notifyPayout function distributes incoming revenue among all attached tokenIds from the previous epoch by consulting the totalBalanceHistory for the previous epoch and using it as divisor for the incoming revenue amount:

> revenueGrowthX128 = amount * X128 / totalBalanceHistory[currentEpochNumber -1]

It is then claimable by multiplying the individual tokenId VP:

> tokenIdVP * revenueGrowthX128 / X128

Core Invariants:

INV 1: Revenue must be distributed among the total VP of the previous epoch

INV 2: Authorized addresses can claim revenue for a tokenId

INV 3: Claiming is only allowed for revenue which has been distributed during _currentEpoch -1

INV 4: Before notifyPayout, the epoch within the Operator must be flipped

Issue_17	Loss of rewards for burned tokenIds
Severity	**High**
Description	Currently, the claim function transfers rewards to the lastOwnerOf address which is compatible with burned tokenIds. However, it uses the wrong modifier which is not compatible with burned tokenIds: *modifier onlyAuthorized(uint256 _tokenId) {* *if (!IVotingEscrow(ve).isAuthorized(msg.sender, _tokenId)) {* *revert NotAuthorized();* *}* *_;* *}* Therefore, rewards for burned tokenIds are permanently lost.
Recommendations	Consider consulting the isAuthorizedAllowBurntToken from the VE.
Comments / Resolution	Resolved.

Issue_18	Incompatibility with transfer-tax tokens
Severity	**Informational**
Description	This contract is not compatible with transfer-tax tokens. If these token types are used for any purpose within the contract, this will result in down-stream issues and inherently break the accounting.
Recommendations	Consider not using these tokens.
Comments / Resolution	Acknowledged.

BribeScheduler

The BribeScheduler contract allows whitelisted addresses, which are determined by governance, to create bribes for the current and future epochs. This can be considered as a simple schedule with the following properties:

> owner: Schedule creator
> bribeVault: Vault that receives the schedule
> token: Token to be distributed
> initialAmount: The initial added amount
> amountLeft: The leftover amount
> startEpochNumber: First epoch
> endEpochNumber: Epoch where schedule stops
> amountAt: Amount for each epoch

Once created, the owner can also cancel it which then distributes the amountLeft value back. The relay function then allows for the permissionless distribution of scheduled bribes in the corresponding epoch.

Core Invariants:

INV 1: amountLeft must be the aggregate of all amountAt values

INV 2: Only the original owner can cancel a schedule

INV 3: A past schedule cannot be canceled

INV 4: Only whitelisted addresses can create schedules

INV 5: Relay must distribute amountAt for current epoch

Privileged Functions

- setWhitelist

Issue_19	Undistributed rewards can never be claimed without deleting the full schedule
Severity	**Low**
Description	Currently, it is only possible to delete a full schedule and claim all outstanding rewards. This can specifically become a problem if for example a distribution is scheduled for one year ahead while one epoch in the beginning remains undistributed. This would essentially lock these funds until either the distribution is fulfilled or canceled and creates opportunity cost.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_20	Deleting schedule will not reset amountAt mapping
Severity	**Low**
Description	Currently, whenever a schedule is removed, this will be done using the delete keyword: *delete _schedules[scheduleId];* This will not work for mappings inside mappings, which is true for the amountAt value.
Recommendations	Consider acknowledging this issue as it is impracticable to loop over all epochs and at the same time there is no harm besides resulting in an incorrect view return value.
Comments / Resolution	Acknowledged.

Issue_21	Incompatibility with transfer-tax tokens
Severity	Informational
Description	This contract is not compatible with transfer-tax tokens. If these token types are used for any purpose within the contract, this will result in down-stream issues and inherently break the accounting.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.